



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Integrantes:

Santiago Zamora

Cristian Moreno

Stephan García

Daniel Garzón

Docente:

Sergio Vargas Pedraza

Asignatura:

Ingeniería de software 2

Universidad Nacional de Colombia
Facultad de ingeniería
Ingeniería de sistemas y computación

12 de junio de 2026

1.Introducción:

ASTRALIS es una plataforma web que permite simular, visualizar y predecir fenómenos astronómicos usando datos reales de la NASA. El sistema integra cuatro módulos técnicos desarrollados por roles independientes que se comunican a través de una API REST central.

El sistema está diseñado con las mismas preocupaciones de cualquier producto profesional en producción: escalabilidad, seguridad, automatización y ética de software.

2. Auditoría de Requerimientos Funcionales (RF)

Estado evaluado contra el código fuente del repositorio entregado (Fase 1):

ID	Descripción	Estado	Observaciones
RF 1	Autenticación (JWT + OAuth2)	Cumple	JWT implementado . OAuth2 como spec de FastAPI . Pero falta OAuth2 con proveedor externo (Google/GitHub). Solo hay registro y login con email/password.
RF 2	API REST ≥ 6 endpoints documentados (Swagger)	Cumple	Swagger en /docs y /redoc . Endpoints: /register, /login, /exoplanets, /exoplanets/{id}, /exoplanets/{id}/lightcurve, /iss/position, /simulation/bodies, /health = 8 endpoints
RF 3	Simulador orbital N-cuerpos (JPL Horizons)	Cumple	Endpoint /simulation/bodies existe con datos medios del JPL . Pero la simulación N-cuerpos en tiempo real (integrador numérico) no está implementada — el Notebook de E3 solo tiene la Ley de Kepler simple.
RF 4	Clasificación ML de exoplanetas (Kepler)	Cumple	El Notebook .ipynb existe con análisis exploratorio .

RF 5	Visualización simulador orbital 2D animado	Cumple	OrbitalCanvas.tsx implementa simulación 2D con Canvas API . Consume /api/simulation/bodies del backend .
RF 6	Catálogo de exoplanetas con filtros	Cumple	Paginación, filtro por classification (CONFIRMED/FALSE_POSITIVE/CANDIDATE), cards individuales .
RF 7	Curvas de luz de tránsito	Cumple	LightCurveChart.tsx + endpoint /lightcurve . Curva generada matemáticamente con forma de tránsito realista y ruido fotométrico.
RF 8	Visualización 3D del sistema solar (Three.js)	Cumple	PlanetViewer.tsx implementa el sistema solar 3D con texturas reales usando Three.js .
RF 9	Pipeline CI/CD (lint → tests → build → deploy)	Cumple	CI en GitHub Actions corre tests del backend y build del frontend . Falta: paso de lint explícito , paso de deploy automatizado .
RF 10	Modelo ML como microservicio con versionado	No cumple	No existe código de microservicio ML en el repo. Solo se referencia en .env como ML_SERVICE_URL. Notebook de análisis presente pero sin servidor.
RF 11	Posición ISS en tiempo real (Open Notify API)	Cumple	Endpoint /api/iss/position hace proxy a Open Notify con timeout y manejo de errores. Frontend ISSMap.tsx muestra mapa en tiempo real con React Leaflet .

Conclusión de Requerimientos Funcionales

De los 11 requerimientos funcionales evaluados, 8 se cumplen satisfactoriamente, 2 tienen cumplimiento parcial (RF3 y RF4) y 1 no cumple (RF10). Los módulos de visualización (E4) y backend (E1) muestran el mayor avance. El rezago principal corresponde al microservicio ML de E3, que es referenciado por el sistema pero no existe como servicio desplegable.

5. Auditoría de Requerimientos No Funcionales (RNF)

ID	Descripción	Estado	Observaciones
RNF-01	Seguridad — OWASP Top 10	Cumple	Headers de seguridad (X-Frame-Options, HSTS, XSS-Protection, CSP vía Permissions-Policy) . Rate limiting en endpoints de auth . bcrypt para passwords . Input validation con Pydantic .
RNF-02	Cobertura de tests $\geq 60\%$	No cumple	Solo 2 tests (test_index y test_health) en test_main.py. No hay tests para auth, exoplanets, ISS ni simulation. Cobertura estimada $< 20\%$.
RNF-03	Monitoreo activo con alertas (UptimeRobot + Grafana)	No cumple	No hay evidencia de configuración de Grafana Cloud, UptimeRobot ni logging centralizado en el repositorio.
RNF-04	Tests de carga con k6/Locust	No cumple	No existe ningún archivo de tests de carga en el repositorio.
RNF-05	UX Responsive	Cumple	Tailwind CSS configurado . La UI usa clases responsive. El frontend fue construido con diseño adaptable.
RNF-06	Ramas protegidas y reglas de PR (monorepo GitHub)	Parcial	El repositorio es un monorepo . El CI corre en PRs a main . Pero la configuración de branch protection rules vive en GitHub settings, no es verificable desde el código — se asume pendiente.
RNF-07	Métricas ML documentadas (F1, accuracy, sesgo)	Parcial	El Notebook tiene análisis exploratorio pero no documenta métricas formales F1/accuracy/matriz de confusión de un modelo entrenado. Pendiente para Fase 3.
RNF-08	Logs centralizados y runbook del pipeline	No cumple	No hay configuración de logging centralizado ni runbook documentado en el repo.
RNF-09	Reflexión ética del modelo ML	No cumple	No existe documento de reflexión ética en el repositorio. Pendiente entregable formal para Fase 4.

RNF-10	Despliegue en la nube con Docker	Parcial	Dockerfiles presentes . docker-compose.yml funcional . Pero falta el servicio de DB en docker-compose y no hay evidencia de deploy a producción (Railway/Render/Fly.io) confirmado.
RNF-11	Integración end-to-end entre los 4 módulos	Parcial	E1 (Backend) ↔ E4 (Frontend) integrados . E2 (DevOps) tiene CI funcionando . E3 (ML/microservicio) no está integrado — el endpoint de clasificación ML no consume ningún modelo real .